

Resumen teórico/práctico — Sistemas Operativos

Notas del curso: *Taller xv6*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico/práctico del capítulo *Taller xv6* de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC). Se describe **xv6**, el SO educativo del MIT en su port a RISC-V usado en el taller: su diseño monolítico, su estructura de fuentes (**kernel/** y **user/**), el *build system* (Makefile, qemu, ELF, **fs.img**), la representación de los procesos (**struct proc**), la arquitectura RISC-V que ejecuta el sistema (ISA modular, modos *machine*, *supervisor* y *user*, registros de control y estado), el mecanismo de *traps* y delegación a modo supervisor, los mapas de memoria y la implementación de tablas de páginas multinivel en Sv39 (con la página *trampoline* mapeada en cada proceso), los traps en modo usuario y, finalmente, la secuencia completa de *boot* (de **_entry** en assembly hasta el primer **exec("sh")**) cuando se ejecuta **make qemu**. El objetivo es servir de material de estudio para el taller xv6 y el final de la materia.

Índice

1. Introducción	2
1.1. Toolchain	2
2. Descripción general	2
3. Diseño y estructura del código	2
3.1. Módulos del kernel	3
4. Build system	3
4.1. Biblioteca estándar de xv6	3
5. Procesos: struct proc	4
5.1. Campos principales	4
6. La arquitectura RISC-V	4
6.1. ISA modular	4
6.2. Modos de ejecución	5
6.3. La placa virtIO emulada por qemu	5
7. Traps y modos de ejecución	5
7.1. Pasos que ejecuta la CPU ante un trap	6
7.2. CSRs en modo supervisor	6
7.3. CSRs en modo machine	6
7.4. Delegación de traps	6
7.5. Transición de modo	6
7.6. Trap handlers en xv6	7
8. Mapas de memoria y protección	7
8.1. Función de traducción	7

8.2. Layout de memoria en qemu-RV39	8
9. Implementación de tablas de páginas en Sv39	8
9.1. Estructura Sv39	8
9.2. MMU y traducción	9
9.3. El registro <code>satp</code>	9
10. Traps en modo usuario	9
10.1. La página trampoline	9
10.2. Secuencia del trap	10
11. Ejecución de xv6 sobre qemu: make qemu	10
11.1. Pasos del boot	10
12. Cierre	11

1. Introducción

xv6 es un sistema operativo simple, tipo UNIX, desarrollado en el curso *Operating System Engineering* del MIT con el objetivo de usar un SO *simple de entender pero realista* en cursos de grado y posgrado. En su versión actual se ha realizado un *port* a la arquitectura **RISC-V**.

1.1. Toolchain

Para compilar xv6 hay que instalar un **cross-compiler** (un compilador que corre en una plataforma pero genera código para otra). Las herramientas usadas son:

- **qemu**: máquina virtual usada como plataforma RISC-V.
- **GNU toolchain para RISC-V**: `gcc` (cross-compiler y linker), `gdb` (debugger), `binutils` (`objdump` y otros).

Idea intuitiva

xv6 corre dentro de una máquina virtual qemu que emula una placa RISC-V (*virtIO board*). El SO huésped se cree corriendo sobre hardware real, pero todo lo que toca son dispositivos virtuales que qemu provee.

2. Descripción general

xv6 está escrito en **C** con pocas líneas de assembly. El objetivo de su diseño e implementación es la **facilidad de lectura y comprensión**, no el rendimiento. Sus características principales son:

- Diseño **monolítico**: kernel en un único espacio de memoria.
- **Mono-usuario** y **multitarea** con preemption.
- Soporta arquitecturas **multicore** o **multiprocesamiento simétrico (SMP)**.
- Ofrece una API que es un *subconjunto* del UNIX v6.

3. Diseño y estructura del código

El código fuente se divide en dos carpetas principales:

- **kernel/**: implementación del núcleo de xv6.
- **user/**: código de los procesos de usuario básicos (comandos, un shell de línea de comandos y un README).

3.1. Módulos del kernel

- **Parámetros del sistema**: constantes sobre tamaños máximos de estructuras (**param.h**).
- **Gestión de procesos**: **proc.c**, **exec.c**.
- **Gestión de memoria**: **kalloc.c**, **vm.c**.
- **Dispositivos de E/S**: soporta básicamente dos dispositivos:
 1. **Terminal** (teclado y pantalla) conectada al puerto serie **UART**, implementada en los archivos **console.c** y **uart.c**.
 2. **Disco** que soporta la interfaz virtIO (**virtio_disk.c**).

En **plic.c** se implementa el control del *RISC-V Platform Level Interrupt Controller (PLIC)*, que permite controlar y rutear interrupciones de dispositivos.

- **Sistema de archivos**: basado en *i-nodes* y **transaccional**. Diseño modular en capas:
 1. Buffer caché y E/S de bloques: **bio.c**.
 2. Logging (transacciones): **log.c**.
 3. I-nodes, directorios, file paths: **fs.c**.
 4. File descriptors: **file.c**.
- **Arquitectura**: soporte para RISC-V en **riscv.h**. Contiene funciones y macros de bajo nivel para control de CPUs y protección (niveles de ejecución), protección y memoria virtual y control de interrupciones y excepciones.

4. Build system

El build system se basa en **make**. El **Makefile** contiene reglas para construir dos productos:

- **kernel**: ejecutable en formato **ELF**, imagen del kernel de xv6.
- **fs.img**: imagen del sistema de archivos en el formato del FS implementado. Internamente contiene los archivos de usuario (programas y datos); representa la imagen del disco para la máquina virtual qemu.

Al ejecutar **make qemu** se recompilan los componentes necesarios, se generan las imágenes y se invoca:

```
qemu-system-riscv64 kernel/kernel fs.img
```

con opciones que indican el número de CPUs (cores) y la cantidad de RAM a usar (ver la regla **qemu: \$K/kernel fs.img** del **Makefile**).

4.1. Biblioteca estándar de xv6

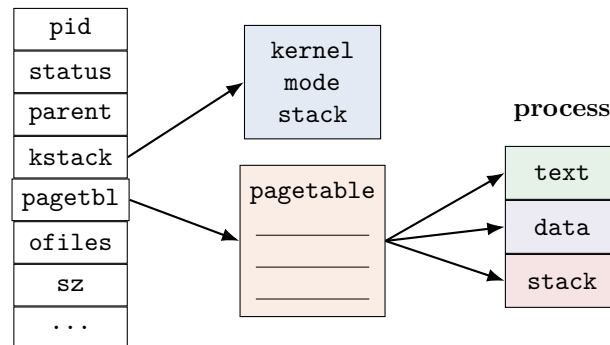
Los programas de usuario (en **user/**) se enlazan con:

- **ulib.c**: funciones de manejo de strings y otros.
- **usys.S**: llamadas al sistema.
- **printf.c**: implementación de **printf()**.
- **umalloc.c**: manejador del heap en modo usuario (**malloc()** / **free()**).

Estos módulos conforman la *biblioteca estándar* de xv6. Las reglas usan `objcopy` y `objdump` para extraer información como la tabla de símbolos y el código assembly de cada objeto para su análisis.

5. Procesos: struct proc

Cada proceso de usuario se representa en el kernel mediante la estructura `struct proc` (definida en `kernel/proc.h`).



5.1. Campos principales

- **parent**: puntero al proceso padre. Cada proceso excepto `init` tiene un padre.
- **kstack**: la **pila en modo kernel**. Se usa cuando ocurre un *trap* (interrupción o excepción) o el proceso realiza una `syscall`: el sistema salta al trap handler del kernel y la CPU cambia de pila.
- **pagetbl**: la **tabla de páginas** del proceso, que define el mapa de memoria (código, datos globales y pila en modo usuario).
- **ofiles**: arreglo con punteros a los descriptores de archivos abiertos por el proceso.
- **sz**: tamaño del espacio (dirección máxima) de memoria usada por el proceso.

xv6 mantiene a los procesos en un **arreglo de dimensión fija** (ver `proc.c`).

6. La arquitectura RISC-V

RISC-V es un *instruction set architecture* (ISA) *abierto*, basado en una arquitectura RISC (*Reduced Instruction Set Computer*). El proyecto fue iniciado en 2010 en la Universidad de California. El objetivo es definir una arquitectura que permita el desarrollo de CPUs compactas, de buen rendimiento y bajo consumo.

6.1. ISA modular

El diseño es modular: parte de un sistema básico con múltiples extensiones posibles.

Nombre	Descripción	Instrucciones
RV32I	Ops enteros 32 bits	40
RV64I	Ops enteros 64 bits	15
RV128I	Ops enteros 128 bits	15

Algunas extensiones:

Nombre	Descripción	Instrucciones
M	Integer multip./division	8 (RV32), 16 (RV64)
A	Atomic instructions	11 (RV32), 13 (RV64)
F	Floating point	26 (RV32), 30 (RV64)
H	Hypervisor instructions	15
S	Supervisor instructions	4

Importante

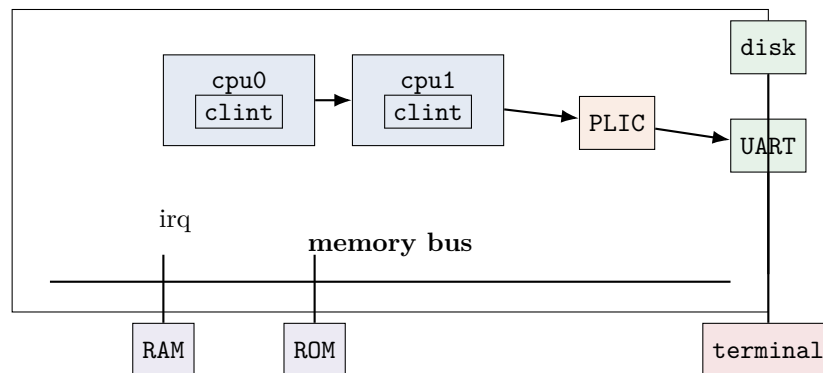
xv6 corre sobre una plataforma **Sv39**: arquitectura de 64 bits con extensiones **M**, **A** y **S**, con un espacio de direcciones de **39 bits**.

6.2. Modos de ejecución

Una CPU RISC-V tiene tres **modos de ejecución**:

- **machine mode**: el modo más privilegiado. La CPU inicia en este modo.
- **supervisor mode**: generalmente usado como *kernel mode*.
- **user mode**: modo no privilegiado.

6.3. La placa virtIO emulada por qemu



Los dispositivos (UART y disco) generan interrupciones que son *routeadas* por el **Platform-Level Interrupt Controller (PLIC)** hacia las CPUs. Cada CPU tiene además un *CLINT* (Core Local Interruptor) para sus interrupciones locales (timer).

7. Traps y modos de ejecución

En cada modo (*machine*, *supervisor*, *user*) existen registros de control y estado (**CSRs**) e instrucciones que sólo pueden accederse o ejecutarse en el modo correspondiente o superior.

Una CPU RISC-V inicia en *machine mode* (mayor privilegio). El SO debe configurar cómo **atrapar los traps**:

- **Interrupciones**: asíncronas, generadas por dispositivos.
- **Excepciones**: *bad address/instruction*, división por cero, etc.
- **Llamadas al sistema**: ejecución de la instrucción `ecall`.

Excepciones y syscalls son **síncronas** porque las genera la CPU al ejecutar `ecall` o al detectar el error.

7.1. Pasos que ejecuta la CPU ante un trap

1. Si es interrupción y `sstatus.SIE == 0` (interrupciones deshabilitadas), se ignora. Si no:
2. Deshabilita interrupciones (0 en el bit `SIE` de `sstatus`).
3. Guarda el program counter `pc` en `sepc`.
4. Guarda el modo corriente en el bit `SPP` de `sstatus`.
5. Setea `scause` con el motivo (código) del trap.
6. Pasa a modo supervisor.
7. Setea `pc = stvec`. En `xv6`, `stvec` apunta a `uservec` o `kernelvec` según si estaba en modo usuario o supervisor.
8. La CPU salta al *interrupt handler* (`uservec`, por ejemplo).

7.2. CSRs en modo supervisor

Nombre	Descripción (uso)
<code>sstatus</code>	Registro de estado
<code>sedeleg</code>	Delegación de excepciones
<code>sideleg</code>	Delegación de interrupciones
<code>sie</code>	Habilitación de interrupciones
<code>stvec</code>	Dirección del trap handler
<code>sscratch</code>	Puntero a datos (parámetros del handler)
<code>sepc</code>	Valor del program counter previo al trap
<code>scause</code>	Causa del trap
<code>stval</code>	Bad address or instruction
<code>sip</code>	Interrupt pending
<code>stp</code>	CPU thread (hart) id
<code>satp</code>	Address translation and protection

7.3. CSRs en modo machine

En *machine mode* los registros anteriores aparecen prefijados con `m` en lugar de `s` (`mstatus`, `mtvec`, `mepc`, ...) y además existen:

Nombre	Descripción
<code>mhartid</code>	Identificador de la CPU (core)
<code>pmpcfg0-15</code>	Physical memory protection: config registers
<code>pmpaddr0-63</code>	Physical memory protection: address registers

Los registros `pmpcfg` y `pmpaddr` permiten configurar **áreas de memoria** y sus modos de acceso en machine mode. En `xv6`, al inicio se configura un área con toda la memoria accesible (ver `start()` en `kernel/start.c`).

7.4. Delegación de traps

Por omisión, *todas* las interrupciones y excepciones se atrapan en machine mode. Se puede configurar la CPU para que las **delegue** a modo supervisor con los registros `medeleg` y `mideleg`. La función `start()` delega los traps a modo supervisor, *excepto* las interrupciones del **timer**, que sólo pueden manejarse en machine mode.

7.5. Transición de modo

Para retornar a un modo menos privilegiado se configura el registro de status (`mstatus` o `sstatus`) y se ejecuta:

- **mret**: retorno desde machine mode.
- **sret**: retorno desde supervisor mode.

Por ejemplo, en xv6, **start()** corre en machine mode y deja **mepc** apuntando a **main** y **mstatus** con MPP en supervisor; al ejecutar **mret** la CPU salta a **main()** en **main.c** corriendo ya en modo supervisor. En cada transición del kernel a un proceso de usuario se configuran **sstatus**, **sepc** y **satp** para que **sret** retorne a modo usuario (ver **usertrapret()** en **trap.c**).

7.6. Trap handlers en xv6

xv6 abstrae interrupciones, excepciones y syscalls como **traps**. Los handlers son:

- **kernelvec**, **timervec** (en **kernelvec.S**): atrapan traps cuando la CPU está en *supervisor mode*.
- **uservec** (en **trampoline.S**): atrapa traps cuando ocurren en *user mode*.

El registro **stvec** contiene la dirección de **kernelvec** o **uservec** según corresponda. Estos handlers salvan el estado de la CPU:

- **kernelvec()** lo hace en la pila actual.
- **uservec()** lo hace en el **trapframe**: un área reservada para cada proceso.

Luego invocan a **kerneltrap()** y **usertrap()** definidas en **trap.c**.

Importante

Una vez iniciado el sistema, las funciones **kerneltrap()** y **usertrap()** son, en la práctica, los **puntos de entrada al kernel**: todo trap (interrupción, excepción o syscall) pasa por ellas.

8. Mapas de memoria y protección

Las arquitecturas modernas proveen mecanismos de memoria virtual y protección. RISC-V particiona la memoria *lógicamente* en **páginas**: bloques de igual tamaño de 4 KB (paginado). Este mecanismo:

- Permite definir *espacios de memoria* para cada proceso y para el kernel.
- Restringe a cada hilo a acceder sólo a sus espacios.
- Permite definir *permisos de acceso* (lectura, escritura, ejecución, ...) por página.

Definición

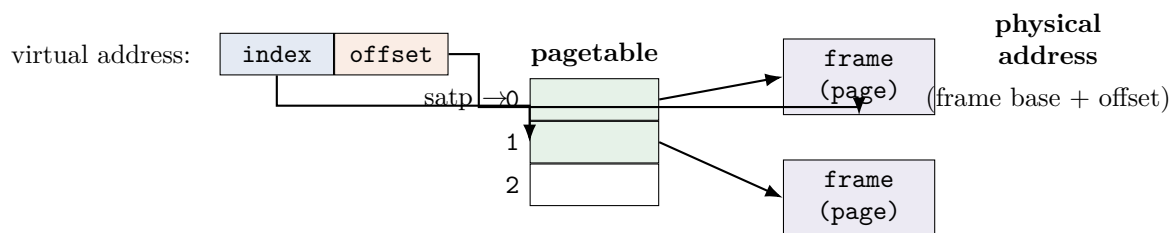
Un **espacio** es un área de memoria *lógicamente contigua* (aunque físicamente sus páginas no necesariamente lo estén).

8.1. Función de traducción

Los espacios se representan en **tablas de páginas** que conceptualmente definen una función

$$mem(virtual_address) \rightarrow physical_address.$$

Las tablas se configuran en memoria como arreglos de punteros a direcciones base de *frames* (o *pages*).



Una dirección lógica se interpreta como un par $(index, offset)$. La dirección física se computa como

$$physical_address = satp[index] + offset.$$

Idea intuitiva

xv6 define una *page table* para el kernel y una *por cada proceso*. La ROM, la RAM y los puertos de los controladores de los dispositivos se mapean en espacios de direcciones físicas de memoria.

8.2. Layout de memoria en qemu-RV39

Rango físico	Contenido
0 – 0x1000	unused
0x1000 – ...	boot ROM
...	...
0x0C000000 – 0x80000000	mmapped devices (PLIC, UART, virtIO disk)
0x80000000 – 0x88000000	physical RAM (kernel + free pages)
$\geq 0x88000000$	free physical space

La tabla de páginas del kernel mapea la RAM y los dispositivos de forma **1 a 1**. El código y datos del módulo `trampoline.S` se mapean en la *última página direccionable* por la arquitectura RV-39, llamada **MAXVA**. Antes de **TRAMPOLINE** (direcciones menores) se mapean las páginas reservadas para el *stack en modo kernel* de cada proceso.

El script `kernel.ld` indica al linker que el código y datos de `trampoline.S` se enlacen a partir de la dirección **TRAMPOLINE**.

La tabla de páginas de un proceso describe su espacio de memoria (código, datos y stack) a partir de la dirección lógica 0 (cero).

9. Implementación de tablas de páginas en Sv39

Una tabla plana sería demasiado grande, por eso se implementa como un **árbol**: cada nodo tiene un número de entradas (*ptes*) tal que el tamaño del nodo coincida con el de una página.

9.1. Estructura Sv39

En Sv39 (64 bits con un bus de direcciones de 39 bits) y páginas de 4 KB, la tabla de páginas es un **árbol de tres niveles**. Una dirección lógica se divide así:

Bits	38–30	29–21	20–12	11–0
Campo	index0	index1	index2	offset

Cada nodo del árbol es de 4 KB y contiene 512 *ptes* de 64 bits de la forma:

63 – 53	53 – 10	9 – 0
not used	page number (pn)	flags

Donde el **page number** corresponde a la dirección física base de la página, y los flags son bits de permisos de acceso:

- **U**: permiso de acceso (si está en 1) en modo usuario.
- **W**: permiso de escritura en la página.
- **V**: si es 1, la *pte* es válida.
- Otros (R, X, A, D, G, ...).

9.2. MMU y traducción

Definición

La **MMU** (*Memory Management Unit*) es el subsistema de la CPU que realiza la traducción de direcciones virtuales a físicas y la verificación de acceso.

Los 9 bits 38–30 indexan en la *raíz*, los bits 29–21 en el *nodo hijo intermedio*, y los bits 20–12 en el *nodo hoja*. La *pte* en la hoja contiene la dirección física base de la página. La MMU computa:

$$pa = ((root[va.index0].pn)[va.index1].pn)[va.index2].pn + va.offset.$$

9.3. El registro satp

En modo supervisor, el registro **satp** (*Supervisor Address Translation and Protection*) apunta a la *raíz* del árbol de la tabla de páginas en uso por la CPU.

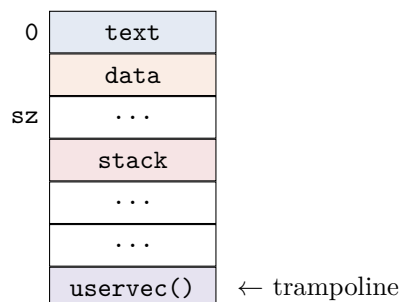
Importante

En cada *context switch*, xv6 debe hacer que **satp** apunte a la tabla de páginas del proceso al que se le da el control. Esto se hace en la función `userret()` (en `trampoline.S`), invocada como `userret(p->pagetable)` desde `usertrapret()` en `trap.c`. `userret` también retorna desde modo supervisor a modo usuario para continuar con la ejecución del proceso corriente.

10. Traps en modo usuario

Al ocurrir un trap en *user mode*, la CPU salta a `uservec()` en `trampoline.S`. Esta función forma parte del código del kernel, pero la **CPU está usando la page table del proceso de usuario**.

10.1. La página trampoline



xv6 **mapea la página física de trampoline** (que contiene `uservec()` y `userret()`) en el espacio de memoria del proceso cuando éste fue creado. Ver `alloc_proc()` y `proc_pagetable()` en `proc.c`.

Idea intuitiva

Esto es necesario porque cuando ocurre una interrupción, excepción o syscall en modo usuario, la CPU *no* cambia automáticamente a la tabla de páginas del kernel (como sí ocurre en CPUs x86). La CPU pasa a ejecutar código del kernel (`uservec()`) en modo supervisor pero en el *espacio de memoria del proceso*; ese código debe ser accesible. Podemos decir que `trampoline.S` contiene **código compartido entre el kernel y todos los procesos**.

10.2. Secuencia del trap

1. La CPU salta a `uservec()`.
2. `uservec()` hace que la CPU use la tabla de páginas *del kernel*.
3. Se ejecuta el código del kernel.
4. Antes de retornar a modo usuario, se restaura el valor de `satp` para que apunte nuevamente a la tabla de páginas del proceso correspondiente.

11. Ejecución de xv6 sobre qemu: make qemu

La regla `make qemu` del Makefile lanza qemu con `kernel` como imagen del kernel y `fs.img` como imagen del disco, con `CPUS` cores y 128 MB de RAM.

qemu carga el kernel en la dirección `0x80000000` (comienzo de la RAM física) y pasa el control a esa dirección. El código del kernel está compilado y enlazado (ver `kernel/kernel.ld`) para ejecutar a partir de allí. La primera función enlazada en el kernel (en `0x80000000`) es `_entry()`, definida en assembly en `entry.S`.

11.1. Pasos del boot

Importante

En RISC-V cada CPU inicia en paralelo en *machine mode*: cada CPU comienza a ejecutar `_entry()`.

1. `_entry()`: prepara cada CPU (*hart* en terminología de RISC-V) para usar un *stack inicial* y invoca a `start()` (en `start.c`).
2. `start()`: configura cada CPU para saltar a `main()` (`main.c`) en *modo supervisor*. Deshabilita paginado (por el momento) y configura las CPUs para que las interrupciones (disco, UART) y excepciones se atrapen en modo supervisor. Aquí también se configura el *timer* (CLINT) para cada CPU y el *timer interrupt handler* (`timervec` en `kernelvec.S`). Las interrupciones del timer se deben atrapar en *machine mode*.
3. `start()` retorna del machine mode ejecutando `mret` y salta a `main()` en modo supervisor.
4. `main()`: hace que la CPU 0 inicialice los subsistemas y cree el primer proceso (ver `userinit()` en `proc.c`), que tiene como código `initcode` (binario extraído de `initcode.S` y *hardcoded* en el arreglo `initcode[]`). Esa secuencia de instrucciones de máquina es equivalente a `exec("init", ["init", 0])`. Finalmente, cada CPU invoca a `scheduler()`.
5. `scheduler()`: las CPUs compiten por seleccionar un proceso en estado `RUNNABLE` y lo pasan a `RUNNING` (le asignan la CPU corriente), haciendo el primer *context switch* a modo usuario. Una de las CPUs encontrará al primer proceso con el código de `initcode`, que tiene un *contexto* salvado de interrupción (creado durante su creación). `scheduler()` cambia al contexto del proceso ejecutando `swtch(&c->context, &p->context)`.

6. La CPU continúa ejecutando `forkret()` (indicado por el valor de `pc` en `p->context`) y finalmente alcanza `usertrapret()`, que ejecuta un *retorno de interrupción* de modo supervisor a modo usuario, restaurando previamente los valores de los registros de la CPU guardados en el `trapframe` en el stack en modo kernel del proceso. En el primer proceso, el `pc` queda apuntando a la primera instrucción del programa `initcode`.
7. El código de `initcode` ejecuta la syscall `exec("init", ...)`. Esta llamada al sistema:
 - **Reemplaza** la imagen de memoria del código y datos del proceso corriente.
 - Carga del disco (filesystem) las instrucciones y datos del ejecutable `init` (ELF).
 - Inicializa el stack en modo usuario (argumentos de `main`).
 - Configura el `trapframe` para que, al retornar de la interrupción, `init` comience en su *entry point* (`main`).
8. El proceso `init` (con `pid = 1`) abre los archivos de entrada y salida estándar y lanza el shell haciendo `fork()` y `exec("sh", ...)`. Este último permite la **interacción del usuario** con el sistema.

A partir de aquí, cada comando que el usuario tipea es procesado por el shell, que realiza las syscalls correspondientes para lanzar nuevos procesos.

Idea intuitiva

Para más detalles del diseño e implementación de xv6, ver el *xv6 book* (manual oficial del MIT).

12. Cierre

xv6 es la encarnación didáctica más simple posible de un SO realista. Las ideas más importantes del taller:

- **Diseño minimalista pero completo:** monolítico, multitarea preemptive, con soporte SMP/multicore y una API subconjunto del UNIX v6. Lo justo para entender qué hace un SO sin perderse en la complejidad de Linux.
- **Estructura clara del código fuente:** `kernel/` y `user/`. Cada subsistema en pocos archivos identificables (gestión de procesos, memoria, dispositivos, FS basado en inodes con *logging* transaccional).
- **Build system simple:** un `Makefile` produce `kernel` (ELF) y `fs.img`. `make qemu` lanza todo dentro de un emulador RISC-V con CPUs y RAM configurables.
- **Procesos representados por `struct proc`:** un arreglo de tamaño fijo, con `pagetbl`, `kstack`, `ofile`s y demás campos esenciales.
- **RISC-V** es una ISA *abierto* y modular con tres modos de privilegio (*machine*, *supervisor*, *user*). xv6 usa el modelo **Sv39**: 64 bits con 39 bits de dirección física.
- **Traps unificados:** interrupciones, excepciones y syscalls. Los pasos automáticos de la CPU (guarda `pc`, modo, causa; pasa a supervisor; salta a `stvec`) se encadenan con `uservec` / `kernelvec` y, en C, con `usertrap()` / `kerneltrap()`, que son en la práctica los puntos de entrada al kernel.
- **Tablas de páginas multinivel** (árbol de 3 niveles en Sv39), apuntadas por `satp`. xv6 mantiene una tabla por proceso y una para el kernel. Una página **trampoline** se mapea en `MAXVA` en todos los espacios para permitir que el kernel atrape traps de modo usuario sin cambiar de tabla automáticamente.

- **Boot determinístico:** `_entry` → `start` (machine) → `main` (supervisor) → `scheduler` → primer context switch a `initcode` → `exec("init")` → `init` abre `stdio` y lanza `sh`. A partir de ese momento, el usuario está al mando.

Idea intuitiva

xv6 cierra el círculo de todo lo visto en la materia: paginado, memoria virtual, scheduling, interrupciones, syscalls, filesystem, drivers, multicore. Su lectura es la mejor forma de *ver* cómo encajan los conceptos del curso en un sistema operativo real, ejecutable y compacto.